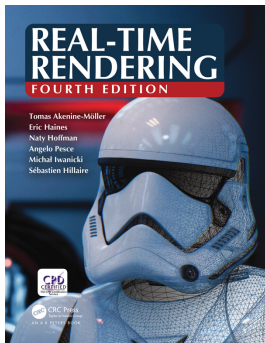


Transformations, Camera and Scenegraph

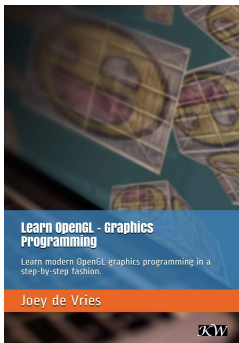
Introduction to Computer Graphics - Lecture 04

Pirmin Pfeifer



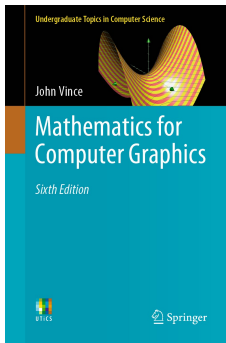
Related Chapters:

- ▶ 4 Transforms



Related Chapters:

- ▶ Part I - 8 Transforms
- ▶ Part I - 9 Coordinate Systems
- ▶ Part I - 11 Camera



Related Chapters:

- ▶ 5 Coordinate Systems
- ▶ 8 Matrix Algebra
- ▶ 10 Geometric Transforms

Motivation



Motivation



To render scenes we need to encode position, rotation and scale of object.

Motivation



To render scenes we need to encode position, rotation and scale of object. The mathematical construct which is very convenient for this is a Matrix.

Matrix

A matrix is an array of numbers organized in m rows and n columns:

A matrix is an array of numbers organized in m rows and n columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}.$$

A matrix is an array of numbers organized in m rows and n columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}.$$

It is also convenient to express the above definition as:

$$A = [a_{ij}]_{mn}$$

Matrix Dimension or Order

The *dimension* or *order* of a matrix is the expression $m \times n$ where m is the number of rows, and n is the number of columns.

Matrix Dimension or Order

The *dimension* or *order* of a matrix is the expression $m \times n$ where m is the number of rows, and n is the number of columns.

Square Matrix

A *square matrix* has the same number of rows as columns:

$$\mathbf{A} = [a_{ij}]_{n \ n} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, \quad \text{e.g.} \quad \begin{bmatrix} 1 & -2 & 4 \\ 6 & 5 & 7 \\ 4 & 3 & 1 \end{bmatrix}.$$

Column Vector

A *column vector* is a matrix with a single column:

$$\begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix}, \quad \text{e.g.} \quad \begin{bmatrix} 2 \\ 3 \\ 23 \end{bmatrix}.$$

Column Vector

A *column vector* is a matrix with a single column:

$$\begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix}, \quad \text{e.g.} \quad \begin{bmatrix} 2 \\ 3 \\ 23 \end{bmatrix}.$$

Row Vector

A *row vector* is a matrix with a single row:

$$[a_{11} \ a_{12} \ \cdots \ a_{1n}], \quad \text{e.g.} \quad [2 \ 3 \ 5].$$

Null Matrix

A *null matrix* has all its elements equal to zero:

$$\theta_n = [a_{ij}]_{n \ n} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}, \quad \text{e.g.} \quad \theta_3 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Null Matrix

A *null matrix* has all its elements equal to zero:

$$\theta_n = [a_{ij}]_{n \ n} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}, \quad \text{e.g.} \quad \theta_3 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Multiplying any matrix with a compatible zero matrix will result in a zero matrix.

Unit Matrix

A *unit matrix* I_n , is a square matrix with the elements on its diagonal a_{11} to a_{nn} equal to 1:

$$\mathbf{I}_n = [a_{ij}]_{n \times n} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}, \quad \text{e.g.} \quad \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Unit Matrix

A *unit matrix* I_n , is a square matrix with the elements on its diagonal a_{11} to a_{nn} equal to 1:

$$\mathbf{I}_n = [a_{ij}]_{n \times n} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}, \quad \text{e.g. } \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The unit matrix behaves like the number 1 in a conventional product, where we have, $1 \times n = n \times 1 = n$, and similarly, $IA = AI = A$.

Transpose

The *transpose* of a matrix exchanges all row elements for column elements. The transposition is indicated by the letter **T** outside the right-hand bracket.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}.$$

$$\begin{bmatrix} 1 & 2 & 4 \\ 6 & 5 & 7 \\ 4 & 3 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 3 \\ 4 & 7 & 1 \end{bmatrix}$$

Transpose

The *transpose* of a matrix exchanges all row elements for column elements. The transposition is indicated by the letter **T** outside the right-hand bracket.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}.$$

Examples:

$$\begin{bmatrix} 1 & 2 & 4 \\ 6 & 5 & 7 \\ 4 & 3 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 3 \\ 4 & 7 & 1 \end{bmatrix}$$

Transpose

The *transpose* of a matrix exchanges all row elements for column elements. The transposition is indicated by the letter **T** outside the right-hand bracket.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}.$$

Examples:

$$\begin{bmatrix} 1 & 2 & 4 \\ 6 & 5 & 7 \\ 4 & 3 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 6 & 4 \\ 2 & 5 & 3 \\ 4 & 7 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}^T = [2 \ 3 \ 5]$$

Matrix Addition and Subtraction

As equations can be added and subtracted together, it follows that matrices can also be added and subtracted, as long as they have the same dimension.

As equations can be added and subtracted together, it follows that matrices can also be added and subtracted, as long as they have the same dimension.

For example, given

$$\mathbf{A} = \begin{bmatrix} 11 & 22 \\ 14 & -15 \\ 27 & 28 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 2 & 1 \\ -4 & 5 \\ 1 & 8 \end{bmatrix}$$

As equations can be added and subtracted together, it follows that matrices can also be added and subtracted, as long as they have the same dimension.

For example, given

$$\mathbf{A} = \begin{bmatrix} 11 & 22 \\ 14 & -15 \\ 27 & 28 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 2 & 1 \\ -4 & 5 \\ 1 & 8 \end{bmatrix}$$

then

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} 13 & 23 \\ 10 & -10 \\ 28 & 36 \end{bmatrix}, \quad \mathbf{A} - \mathbf{B} = \begin{bmatrix} 9 & 21 \\ 18 & -20 \\ 26 & 20 \end{bmatrix}.$$

Scalar Multiplication

As equations can be scaled and factorised, it follows that matrixes can also be scaled and factorised.

$$\lambda \mathbf{A} = \lambda \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} = \begin{bmatrix} \lambda a_{11} & \lambda a_{12} & \dots & \lambda a_{1n} \\ \lambda a_{21} & \lambda a_{22} & \dots & \lambda a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda a_{m1} & \lambda a_{m2} & \dots & \lambda a_{mn} \end{bmatrix}.$$

Scalar Multiplication

As equations can be scaled and factorised, it follows that matrixes can also be scaled and factorised.

$$\lambda \mathbf{A} = \lambda \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} = \begin{bmatrix} \lambda a_{11} & \lambda a_{12} & \dots & \lambda a_{1n} \\ \lambda a_{21} & \lambda a_{22} & \dots & \lambda a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda a_{m1} & \lambda a_{m2} & \dots & \lambda a_{mn} \end{bmatrix}.$$

For example,

$$2 \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}.$$

Row and Column Vectors

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix}$$

Row and Column Vectors

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = a\alpha + b\beta + c\gamma$$

Row and Column Vectors

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = a\alpha + b\beta + c\gamma$$

which is a scalar and equivalent to the dot or scalar product of two vectors.

Row Vector and a Matrix

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix}$$

Row Vector and a Matrix

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix}$$

Row Vector and a Matrix

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix}$$

then,

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix} \\ &= \begin{bmatrix} (a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}) & (a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32}) & (a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33}) \end{bmatrix}. \end{aligned}$$

Row Vector and a Matrix

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix}$$

then,

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} a_{11} & a_{12} & a_{13} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{m1} & b_{m2} & b_{33} \end{bmatrix} \\ &= \begin{bmatrix} (a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31}) & (a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32}) & (a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33}) \end{bmatrix}. \end{aligned}$$

The product BA is not permitted.

Row Vector and a Matrix - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

Row Vector and a Matrix - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

Row Vector and a Matrix - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

then,

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \\ &= \begin{bmatrix} (2 + 9 + 16) & (4 + 12 + 20) & (6 + 15 + 24) \end{bmatrix} \end{aligned}$$

Row Vector and a Matrix - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

then,

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \\ &= \begin{bmatrix} (2 + 9 + 16) & (4 + 12 + 20) & (6 + 15 + 24) \end{bmatrix} \\ &= \begin{bmatrix} 27 & 36 & 45 \end{bmatrix}. \end{aligned}$$

Matrix and a Column Vector

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix}$$

Matrix and a Column Vector

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}.$$

Matrix and a Column Vector

Given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}.$$

The product BA is not permitted.

Matrix and a Column Vector - *Example*

For example, given

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 + 6 + 12 \\ 6 + 12 + 20 \\ 8 + 15 + 24 \end{bmatrix} = \begin{bmatrix} 20 \\ 38 \\ 47 \end{bmatrix}.$$

Matrix and a Column Vector - *Example*

For example, given

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 + 6 + 12 \\ 6 + 12 + 20 \\ 8 + 15 + 24 \end{bmatrix} = \begin{bmatrix} 20 \\ 38 \\ 47 \end{bmatrix}.$$

then,

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 + 6 + 12 \\ 6 + 12 + 20 \\ 8 + 15 + 24 \end{bmatrix} = \begin{bmatrix} 20 \\ 38 \\ 47 \end{bmatrix}.$$

Square Matrices

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix}$$

Square Matrices

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix} = \begin{bmatrix} a\alpha + b\lambda + c\rho & a\beta + b\mu + c\sigma & a\gamma + b\nu + c\tau \\ p\alpha + q\lambda + r\rho & p\beta + q\mu + r\sigma & p\gamma + q\nu + r\tau \\ u\alpha + v\lambda + w\rho & u\beta + v\mu + w\sigma & u\gamma + v\nu + w\tau \end{bmatrix}$$

Square Matrices

Given

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix} = \begin{bmatrix} a\alpha + b\lambda + c\rho & a\beta + b\mu + c\sigma & a\gamma + b\nu + c\tau \\ p\alpha + q\lambda + r\rho & p\beta + q\mu + r\sigma & p\gamma + q\nu + r\tau \\ u\alpha + v\lambda + w\rho & u\beta + v\mu + w\sigma & u\gamma + v\nu + w\tau \end{bmatrix}$$

and,

$$\mathbf{BA} = \begin{bmatrix} \alpha & \beta & \gamma \\ \lambda & \mu & \nu \\ \rho & \sigma & \tau \end{bmatrix} \begin{bmatrix} a & b & c \\ p & q & r \\ u & v & w \end{bmatrix} = \begin{bmatrix} \alpha a + \beta p + \gamma u & \alpha b + \beta q + \gamma v & \alpha c + \beta r + \gamma w \\ \lambda a + \mu p + \nu u & \lambda b + \mu q + \nu v & \lambda c + \mu r + \nu w \\ \rho a + \sigma p + \tau u & \rho b + \sigma q + \tau v & \rho c + \sigma r + \tau w \end{bmatrix}$$

Square Matrices - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 5 & 6 & 7 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 6 \\ 6 & 7 & 8 \end{bmatrix}$$

Square Matrices - *Example*

For example, given

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 5 & 6 & 7 \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 6 \\ 6 & 7 & 8 \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \\ 5 & 6 & 7 \end{bmatrix} \begin{bmatrix} 2 & 3 & 4 \\ 4 & 5 & 6 \\ 6 & 7 & 8 \end{bmatrix} = \begin{bmatrix} 28 & 34 & 40 \\ 52 & 64 & 76 \\ 76 & 92 & 112 \end{bmatrix}$$

Rectangular Matrices

Given two rectangular matrices **A** and **B**,

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$,

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted,

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted, if and only if, **B** has a dimension $n \times p$.

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted, if and only if, **B** has a dimension $n \times p$. The resulting matrix has a dimension $m \times p$.

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted, if and only if, **B** has a dimension $n \times p$. The resulting matrix has a dimension $m \times p$. For example, given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix}$$

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted, if and only if, **B** has a dimension $n \times p$. The resulting matrix has a dimension $m \times p$. For example, given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix}$$

then,

$$\mathbf{AB} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix}$$

Rectangular Matrices

Given two rectangular matrices **A** and **B**, where **A** has a dimension $m \times n$, the product **AB** is permitted, if and only if, **B** has a dimension $n \times p$. The resulting matrix has a dimension $m \times p$. For example, given

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \quad \text{and} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix}$$

then,

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix} \\ &= \begin{bmatrix} (a_{11}b_{11} + a_{12}b_{21}) & (a_{11}b_{12} + a_{12}b_{22}) & (a_{11}b_{13} + a_{12}b_{23}) & (a_{11}b_{14} + a_{12}b_{24}) \\ (a_{21}b_{11} + a_{22}b_{21}) & (a_{21}b_{12} + a_{22}b_{22}) & (a_{21}b_{13} + a_{22}b_{23}) & (a_{21}b_{14} + a_{22}b_{24}) \\ (a_{31}b_{11} + a_{32}b_{21}) & (a_{31}b_{12} + a_{32}b_{22}) & (a_{31}b_{13} + a_{32}b_{23}) & (a_{31}b_{14} + a_{32}b_{24}) \end{bmatrix}. \end{aligned}$$

Inverse Matrix

A square matrix A_{nn} that is *invertible* satisfies the condition:

Inverse Matrix

A square matrix A_{nn} that is *invertible* satisfies the condition:

$$A_{nn}A_{nn}^{-1} = A_{nn}^{-1}A_{nn} = I_n$$

A square matrix A_{nn} that is *invertible* satisfies the condition:

$$A_{nn}A_{nn}^{-1} = A_{nn}^{-1}A_{nn} = I_n$$

A_{nn}^{-1} is unique, and is the *inverse matrix* of A_{nn} .

A square matrix A_{nn} that is *invertible* satisfies the condition:

$$A_{nn}A_{nn}^{-1} = A_{nn}^{-1}A_{nn} = I_n$$

A_{nn}^{-1} is unique, and is the *inverse matrix* of A_{nn} . For example, given

$$\mathbf{A} = \begin{bmatrix} 4 & 3 \\ 5 & 4 \end{bmatrix}$$

A square matrix A_{nn} that is *invertible* satisfies the condition:

$$A_{nn}A_{nn}^{-1} = A_{nn}^{-1}A_{nn} = I_n$$

A_{nn}^{-1} is unique, and is the *inverse matrix* of A_{nn} . For example, given

$$\mathbf{A} = \begin{bmatrix} 4 & 3 \\ 5 & 4 \end{bmatrix}$$

then,

$$\mathbf{A}^{-1} = \begin{bmatrix} 4 & -3 \\ -5 & 4 \end{bmatrix}$$

A square matrix A_{nn} that is *invertible* satisfies the condition:

$$A_{nn}A_{nn}^{-1} = A_{nn}^{-1}A_{nn} = I_n$$

A_{nn}^{-1} is unique, and is the *inverse matrix* of A_{nn} . For example, given

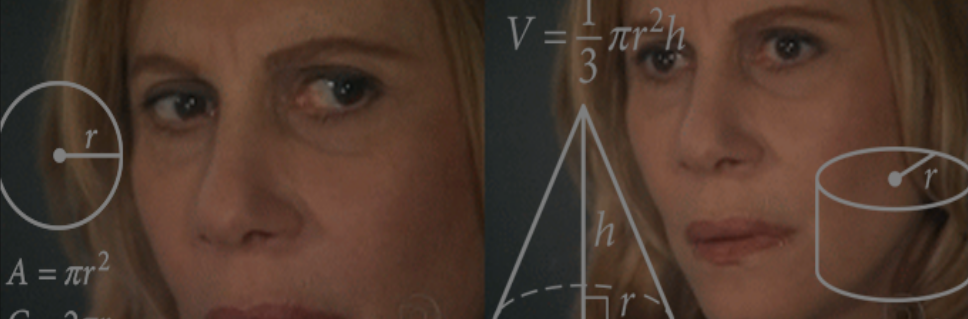
$$\mathbf{A} = \begin{bmatrix} 4 & 3 \\ 5 & 4 \end{bmatrix}$$

then,

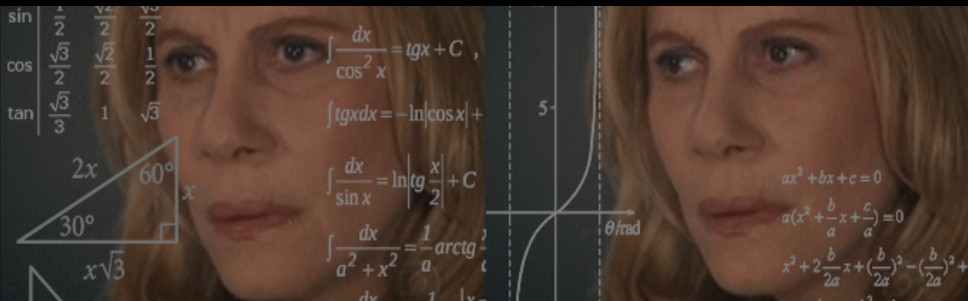
$$\mathbf{A}^{-1} = \begin{bmatrix} 4 & -3 \\ -5 & 4 \end{bmatrix}$$

because,

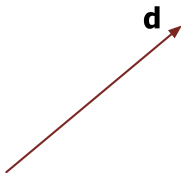
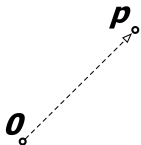
$$\mathbf{A}\mathbf{A}^{-1} = \begin{bmatrix} 4 & 3 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} 4 & -3 \\ -5 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

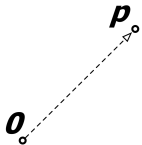


Any Questions?



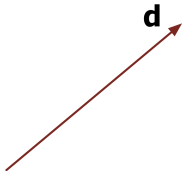
Interpretation of Vectors:

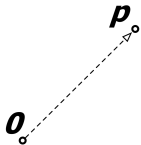




Interpretation of Vectors:

- ▶ Position vector
- ▶ Direction vector

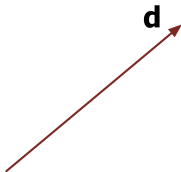


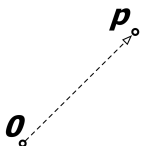


Interpretation of Vectors:

- ▶ Position vector
- ▶ Direction vector

Position Vector:



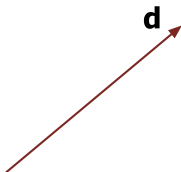


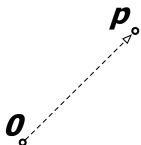
Interpretation of Vectors:

- ▶ Position vector
- ▶ Direction vector

Position Vector:

- ▶ Determines a position in space



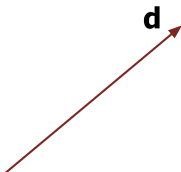


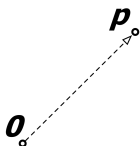
Interpretation of Vectors:

- ▶ Position vector
- ▶ Direction vector

Position Vector:

- ▶ Determines a position in space
- ▶ Always relative to the origin





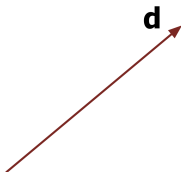
Interpretation of Vectors:

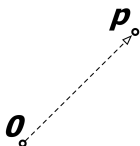
- ▶ Position vector
- ▶ Direction vector

Position Vector:

- ▶ Determines a position in space
- ▶ Always relative to the origin

Direction Vector:





Interpretation of Vectors:

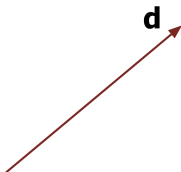
- ▶ Position vector
- ▶ Direction vector

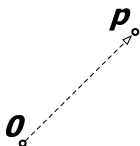
Position Vector:

- ▶ Determines a position in space
- ▶ Always relative to the origin

Direction Vector:

- ▶ Determines a direction





Interpretation of Vectors:

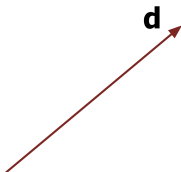
- ▶ Position vector
- ▶ Direction vector

Position Vector:

- ▶ Determines a position in space
- ▶ Always relative to the origin

Direction Vector:

- ▶ Determines a direction
- ▶ Absolute quantity



Homogeneous Representation

Uniformly represent vectors and points in $\mathbb{R}^n$ using a matrix $M_{n+1,1}$.

Homogeneous Representation

Uniformly represent vectors and points in $\mathbb{R}^n$ using a matrix $M_{n+1,1}$.

We differentiate between points and directions through the additional component.

Homogeneous Representation

Uniformly represent vectors and points in $\mathbb{R}^n$ using a matrix $M_{n+1,1}$.

We differentiate between points and directions through the additional component.

Vector v in $\mathbb{R}^3$

$$v = \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix}$$

Homogeneous Representation

Uniformly represent vectors and points in $\mathbb{R}^n$ using a matrix $M_{n+1,1}$.

We differentiate between points and directions through the additional component.

Vector v in $\mathbb{R}^3$

$$v = \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix}$$

Point p in $\mathbb{R}^3$

$$p = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Point-point difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} - \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_1 - s_1 \\ r_2 - s_2 \\ r_3 - s_3 \\ 0 \end{bmatrix}$$

Point-point difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} - \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_1 - s_1 \\ r_2 - s_2 \\ r_3 - s_3 \\ 0 \end{bmatrix}$$

Vector-vector sum/difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 0 \end{bmatrix} \pm \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r_1 \pm s_1 \\ r_2 \pm s_2 \\ r_3 \pm s_3 \\ 0 \end{bmatrix}$$

Point-point difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} - \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_1 - s_1 \\ r_2 - s_2 \\ r_3 - s_3 \\ 0 \end{bmatrix}$$

Vector-vector sum/difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 0 \end{bmatrix} \pm \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r_1 \pm s_1 \\ r_2 \pm s_2 \\ r_3 \pm s_3 \\ 0 \end{bmatrix}$$

Point-vector sum/difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} \pm \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r_1 \pm s_1 \\ r_2 \pm s_2 \\ r_3 \pm s_3 \\ 1 \end{bmatrix}$$

Point-point difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} - \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 1 \end{bmatrix} = \begin{bmatrix} r_1 - s_1 \\ r_2 - s_2 \\ r_3 - s_3 \\ 0 \end{bmatrix}$$

Vector-vector sum/difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 0 \end{bmatrix} \pm \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r_1 \pm s_1 \\ r_2 \pm s_2 \\ r_3 \pm s_3 \\ 0 \end{bmatrix}$$

Point-vector sum/difference

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ 1 \end{bmatrix} \pm \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r_1 \pm s_1 \\ r_2 \pm s_2 \\ r_3 \pm s_3 \\ 1 \end{bmatrix}$$

Scalar-vector product

$$r \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ 0 \end{bmatrix} = \begin{bmatrix} r s_1 \\ r s_2 \\ r s_3 \\ 0 \end{bmatrix}$$

Affine Transformation

In Euclidean geometry, an *affine transformation* is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.

Affine Transformation

In Euclidean geometry, an *affine transformation* is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.

The three most important affine transformations are:

Affine Transformation

In Euclidean geometry, an *affine transformation* is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.

The three most important affine transformations are:

- Translation

Affine Transformation

In Euclidean geometry, an *affine transformation* is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.

The three most important affine transformations are:

- ▶ Translation
- ▶ Rotation

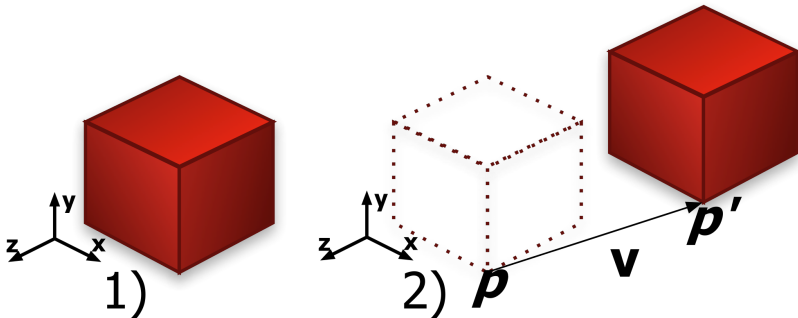
Affine Transformation

In Euclidean geometry, an *affine transformation* is a geometric transformation that preserves lines and parallelism, but not necessarily Euclidean distances and angles.

The three most important affine transformations are:

- ▶ Translation
- ▶ Rotation
- ▶ Scale

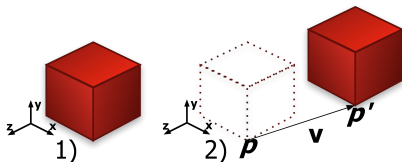
Objective: Move all points $p \in P$ of an object along the displacement vector v .



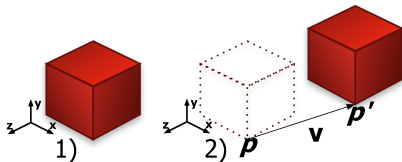
Objective: Move all points $p \in P$ of an object along the displacement vector v .

Given:

- Displacement vector
$$v = \begin{bmatrix} v_x & v_y & v_z & 0 \end{bmatrix}^T$$



Objective: Move all points $p \in P$ of an object along the displacement vector v .



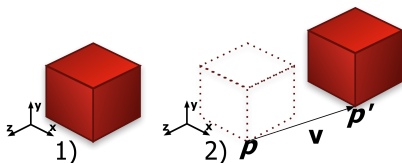
Given:

- Displacement vector
$$v = \begin{bmatrix} v_x & v_y & v_z & 0 \end{bmatrix}^T$$
- Point
$$\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix}^T$$

Objective: Move all points $p \in P$ of an object along the displacement vector v .

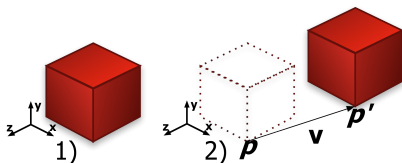
Given:

- Displacement vector
 $v = [v_x \quad v_y \quad v_z \quad 0]^T$
- Point
 $[p_x \quad p_y \quad p_z \quad 1]^T$



$$p' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{bmatrix} = \begin{bmatrix} p_x + v_x \\ p_y + v_y \\ p_z + v_z \\ 1 + 0 \end{bmatrix} = T(v_x, v_y, v_z)p = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Move all points $p \in P$ of an object along the displacement vector v .

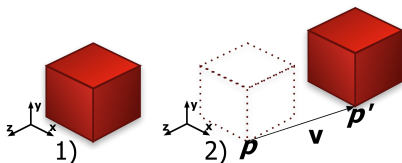


Given:

- Displacement vector
 $v = [v_x \quad v_y \quad v_z \quad 0]^T$
- Point
 $[p_x \quad p_y \quad p_z \quad 1]^T$

$$p' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{bmatrix} = \begin{bmatrix} p_x + v_x \\ p_y + v_y \\ p_z + v_z \\ 1 + 0 \end{bmatrix} = T(v_x, v_y, v_z)p = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Move all points $p \in P$ of an object along the displacement vector v .



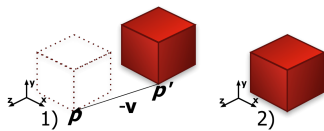
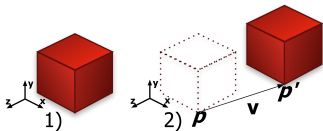
Given:

- Displacement vector
 $v = [v_x \quad v_y \quad v_z \quad 0]^T$
- Point
 $[p_x \quad p_y \quad p_z \quad 1]^T$

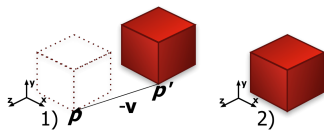
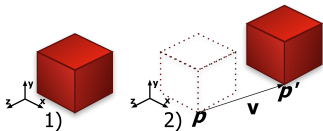
Affine Transform: Translation:

$$p' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{bmatrix} = \begin{bmatrix} p_x + v_x \\ p_y + v_y \\ p_z + v_z \\ 1 + 0 \end{bmatrix} = T(v_x, v_y, v_z)p = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Translation Inverse

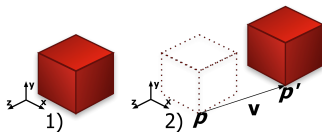


Translation Inverse

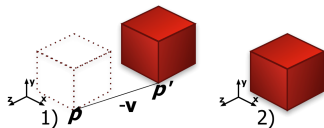


$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translation Inverse

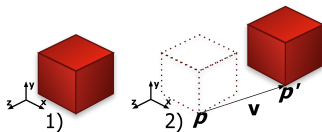


$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

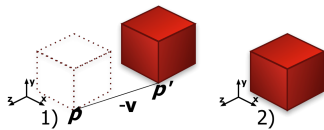


$$T^{-1}(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & -v_x \\ 0 & 1 & 0 & -v_y \\ 0 & 0 & 1 & -v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Translation Inverse



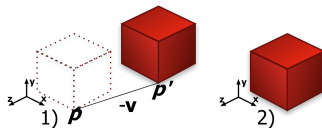
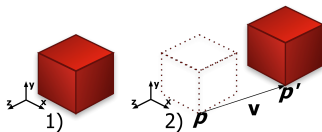
$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$T^{-1}(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & -v_x \\ 0 & 1 & 0 & -v_y \\ 0 & 0 & 1 & -v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

To invert the translation we generate a new translation with negated v :

Translation Inverse



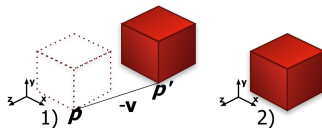
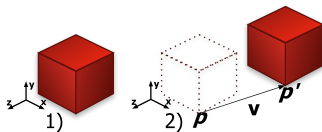
$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T^{-1}(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & -v_x \\ 0 & 1 & 0 & -v_y \\ 0 & 0 & 1 & -v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

To invert the translation we generate a new translation with negated v :

$$T^{-1}(v_x, v_y, v_z) = T(-v_x, -v_y, -v_z)$$

Translation Inverse



$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

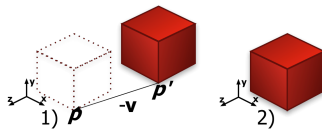
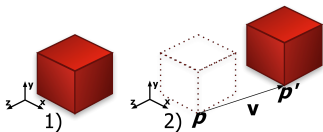
$$T^{-1}(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & -v_x \\ 0 & 1 & 0 & -v_y \\ 0 & 0 & 1 & -v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

To invert the translation we generate a new translation with negated v :

$$T^{-1}(v_x, v_y, v_z) = T(-v_x, -v_y, -v_z)$$

We can revert the the translation with the inverse translation:

Translation Inverse



$$T(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & v_x \\ 0 & 1 & 0 & v_y \\ 0 & 0 & 1 & v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T^{-1}(v_x, v_y, v_z) = \begin{bmatrix} 1 & 0 & 0 & -v_x \\ 0 & 1 & 0 & -v_y \\ 0 & 0 & 1 & -v_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

To invert the translation we generate a new translation with negated v :

$$T^{-1}(v_x, v_y, v_z) = T(-v_x, -v_y, -v_z)$$

We can revert the the translation with the inverse translation:

$$T(v_x, v_y, v_z)T^{-1}(v_x, v_y, v_z) = T^{-1}(v_x, v_y, v_z)T(v_x, v_y, v_z) = I_4$$

Translation Properties

For all $v, w \in V$ holds true:

Translation Properties

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

Translation Properties

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

Translation Properties

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

$$T(v)T(w) = T(v + w)$$

Translation Properties

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

$$T(v)T(w) = T(v + w)$$

$$T(v)T(w) = T(w)T(v)$$

Translation Properties

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

$$T(v)T(w) = T(v + w)$$

$$T(v)T(w) = T(w)T(v)$$

Additionally, the Translation dose not effect:

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

$$T(v)T(w) = T(v + w)$$

$$T(v)T(w) = T(w)T(v)$$

Additionally, the Translation dose not effect:

- The shape of an object

For all $v, w \in V$ holds true:

$$T(0) = I_4$$

$$T^{-1}(v) = T(-v)$$

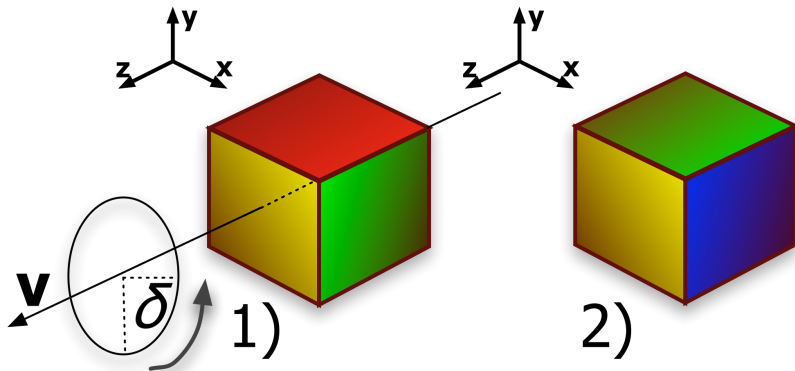
$$T(v)T(w) = T(v + w)$$

$$T(v)T(w) = T(w)T(v)$$

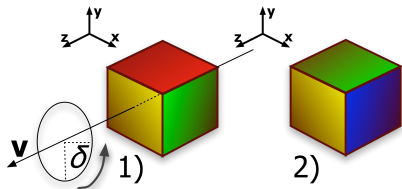
Additionally, the Translation dose not effect:

- ▶ The shape of an object
- ▶ Directions

Objective: Rotate all points $p \in P$ of an object around an axis.



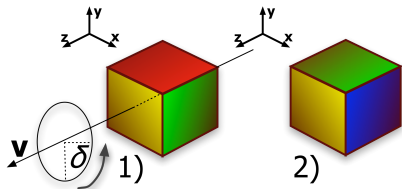
Objective: Rotate all points $p \in P$ of an object around an axis.



Given:

► Axis

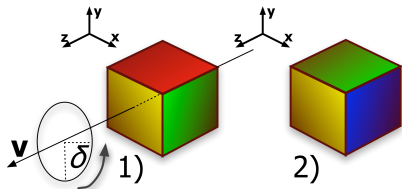
Objective: Rotate all points $p \in P$ of an object around an axis.



Given:

- Axis
- Rotation angle

Objective: Rotate all points $p \in P$ of an object around an axis.



Given:

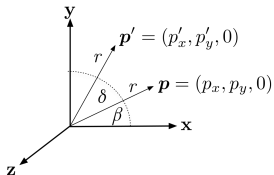
- ▶ Axis
- ▶ Rotation angle
- ▶ Rotation direction

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

Rotation around z I

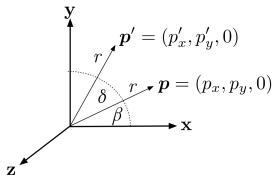
Objective: Rotation of Degree δ of Point p around z



Rotation around z I

Objective: Rotation of Degree δ of Point p around z

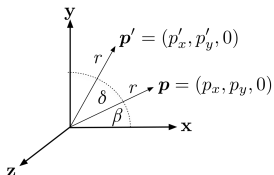
From Trigonometry follows:



Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:

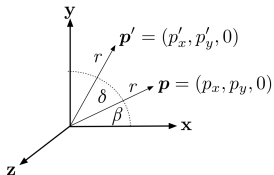


$$\cos(\beta) = \frac{p_x}{r}$$

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



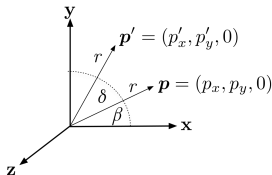
$$\cos(\beta) = \frac{p_x}{r}$$

$$\sin(\beta) = \frac{p_y}{r}$$

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



$$\cos(\beta) = \frac{p_x}{r}$$

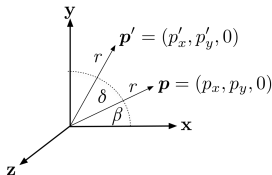
$$\sin(\beta) = \frac{p_y}{r}$$

$$\cos(\beta + \delta) = \frac{p'_x}{r}$$

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



$$\cos(\beta) = \frac{p_x}{r}$$

$$\sin(\beta) = \frac{p_y}{r}$$

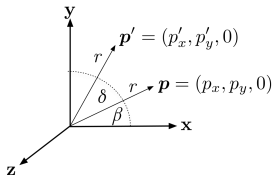
$$\cos(\beta + \delta) = \frac{p'_x}{r}$$

$$\sin(\beta + \delta) = \frac{p'_y}{r}$$

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



$$\cos(\beta) = \frac{p_x}{r}$$

$$\sin(\beta) = \frac{p_y}{r}$$

$$\cos(\beta + \delta) = \frac{p'_x}{r}$$

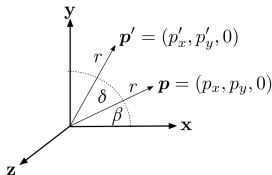
$$\sin(\beta + \delta) = \frac{p'_y}{r}$$

Sum formula:

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



$$\cos(\beta) = \frac{p_x}{r}$$

$$\sin(\beta) = \frac{p_y}{r}$$

$$\cos(\beta + \delta) = \frac{p'_x}{r}$$

$$\sin(\beta + \delta) = \frac{p'_y}{r}$$

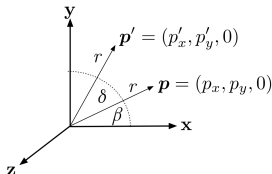
Sum formula:

$$\cos(\beta + \delta) = \cos(\beta)\cos(\delta) - \sin(\beta)\sin(\delta)$$

Rotation around z I

Objective: Rotation of Degree δ of Point p around z

From Trigonometry follows:



$$\cos(\beta) = \frac{p_x}{r}$$

$$\sin(\beta) = \frac{p_y}{r}$$

$$\cos(\beta + \delta) = \frac{p'_x}{r}$$

$$\sin(\beta + \delta) = \frac{p'_y}{r}$$

Sum formula:

$$\cos(\beta + \delta) = \cos(\beta)\cos(\delta) - \sin(\beta)\sin(\delta)$$

$$\sin(\beta + \delta) = \sin(\beta)\cos(\delta) + \cos(\beta)\sin(\delta)$$

Rotation around z II

Therefore follows for p'_x :

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

And for p'_y :

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

And for p'_y :

$$\frac{p'_y}{r} = \frac{p_y}{r} \cos(\delta) + \frac{p_x}{r} \sin(\delta)$$

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

And for p'_y :

$$\frac{p'_y}{r} = \frac{p_y}{r} \cos(\delta) + \frac{p_x}{r} \sin(\delta)$$

$$p'_y = p_y \cos(\delta) + p_x \sin(\delta)$$

Therefore follows for p'_x :

$$\frac{p'_x}{r} = \frac{p_x}{r} \cos(\delta) - \frac{p_y}{r} \sin(\delta)$$

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

And for p'_y :

$$\frac{p'_y}{r} = \frac{p_y}{r} \cos(\delta) + \frac{p_x}{r} \sin(\delta)$$

$$p'_y = p_y \cos(\delta) + p_x \sin(\delta)$$

And for p'_z :

$$p'_z = p_z$$

Given:

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

$$p'_y = p_y \cos(\delta) + p_x \sin(\delta)$$

$$p'_z = p_z$$

Z-Rotation Matrix:

Given:

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

$$p'_y = p_y \cos(\delta) + p_x \sin(\delta)$$

$$p'_z = p_z$$

Z-Rotation Matrix:

$$p' = \begin{bmatrix} p_x \cos(\delta) - p_y \sin(\delta) \\ p_y \cos(\delta) + p_x \sin(\delta) \\ p_z \\ 1 \end{bmatrix} = R_z(\delta) p = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Given:

$$p'_x = p_x \cos(\delta) - p_y \sin(\delta)$$

$$p'_y = p_y \cos(\delta) + p_x \sin(\delta)$$

$$p'_z = p_z$$

Z-Rotation Matrix:

$$p' = \begin{bmatrix} p_x \cos(\delta) - p_y \sin(\delta) \\ p_y \cos(\delta) + p_x \sin(\delta) \\ p_z \\ 1 \end{bmatrix} = R_z(\delta) p = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Given:

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Given:

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Inverse:

$$R_z^{-1}(\delta) = \quad =$$

Given:

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Inverse:

$$R_z^{-1}(\delta) = R_z(-\delta) =$$

Given:

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Inverse:

$$R_z^{-1}(\delta) = R_z(-\delta) = \begin{bmatrix} \cos(-\delta) & -\sin(-\delta) & 0 & 0 \\ \sin(-\delta) & \cos(-\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation Properties

$$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$$

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$$R_a^{-1}(\delta) = R(-\delta) = R_a^T(\delta)$$

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$$R_a^{-1}(\delta) = R(-\delta) = R_a^T(\delta)$$

$$R_a(\delta)R_a(\gamma) = R_a(\delta + \gamma)$$

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$$R_a^{-1}(\delta) = R(-\delta) = R_a^T(\delta)$$

$$R_a(\delta)R_a(\gamma) = R_a(\delta + \gamma)$$

$$R_a(\delta)R_b(\gamma) \neq R_b(\gamma)R_a(\delta)$$

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$$R_a^{-1}(\delta) = R(-\delta) = R_a^T(\delta)$$

$$R_a(\delta)R_a(\gamma) = R_a(\delta + \gamma)$$

$$R_a(\delta)R_b(\gamma) \neq R_b(\gamma)R_a(\delta)$$

Rotations effect points and directions

$\forall v, w \in V; \gamma, \delta \in [0, 2\pi]; a, b \in x, y, z | a \neq b:$

$$R_a(0) = I_4$$

$$R_a^{-1}(\delta) = R(-\delta) = R_a^T(\delta)$$

$$R_a(\delta)R_a(\gamma) = R_a(\delta + \gamma)$$

$$R_a(\delta)R_b(\gamma) \neq R_b(\gamma)R_a(\delta)$$

Rotations effect points and directions , but do not change the shape of an object.

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\delta) = \begin{bmatrix} \cos(\delta) & 0 & \sin(\delta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\delta) & 0 & \cos(\delta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_x(\delta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\delta) & -\sin(\delta) & 0 \\ 0 & \sin(\delta) & \cos(\delta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\delta) = \begin{bmatrix} \cos(\delta) & 0 & \sin(\delta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\delta) & 0 & \cos(\delta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\delta) = \begin{bmatrix} \cos(\delta) & -\sin(\delta) & 0 & 0 \\ \sin(\delta) & \cos(\delta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The Euler Transform

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The Euler Transform

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- roll - rotation around Z / forward axis

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis
- ▶ yaw¹ - rotation around Y / up axis

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis
- ▶ yaw¹ - rotation around Y / up axis

The inverse of the euler transform can easily be computed:

$$E^{-1} =$$

¹sometimes also called head

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis
- ▶ yaw¹ - rotation around Y / up axis

The inverse of the euler transform can easily be computed:

$$E^{-1} = E^T =$$

¹sometimes also called head

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis
- ▶ yaw¹ - rotation around Y / up axis

The inverse of the euler transform can easily be computed:

$$E^{-1} = E^T = (R_z R_x R_y)^T =$$

¹sometimes also called head

The Euler Transform rotates around all three axis:

$$E(p, y, r) = R_z(r)R_x(p)R_y(y)$$

The angles are commonly named:

- ▶ roll - rotation around Z / forward axis
- ▶ pitch - rotation around X / right axis
- ▶ yaw¹ - rotation around Y / up axis

The inverse of the euler transform can easily be computed:

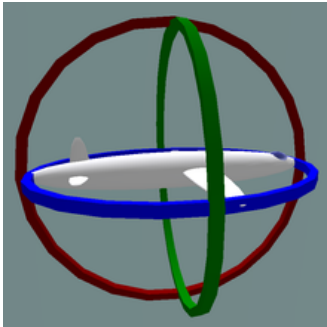
$$E^{-1} = E^T = (R_z R_x R_y)^T = R_y^T R_x^T R_z^T$$

¹sometimes also called head

When using the euler angles as rotation transform,
Gimbal Lock can occur:

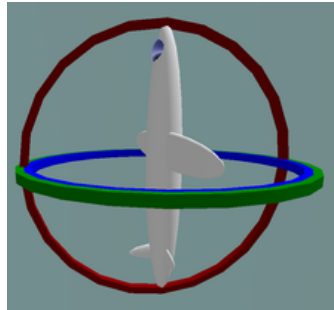
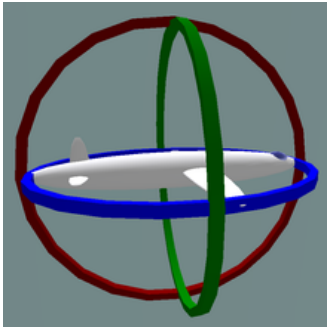
Gimbal Lock

When using the euler angles as rotation transform,
Gimbal Lock can occur:



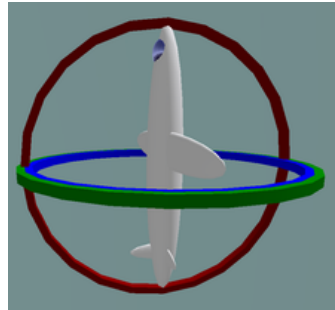
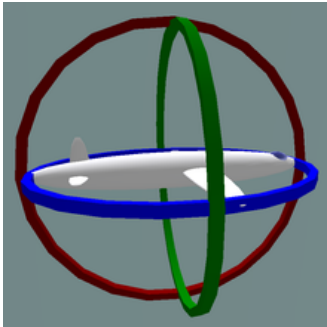
Gimbal Lock

When using the euler angles as rotation transform,
Gimbal Lock can occur:



Gimbal Lock

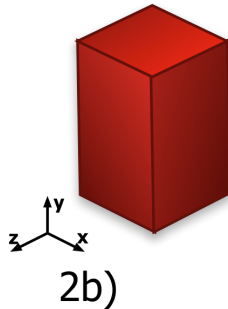
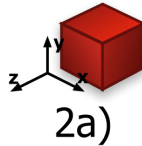
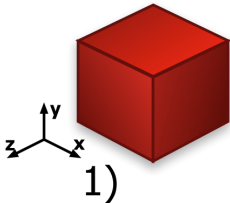
When using the euler angles as rotation transform, Gimbal Lock can occur:



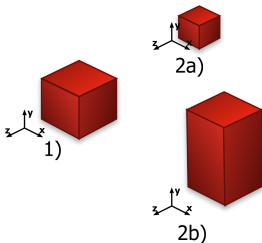
The problem can be circumvented by using *quaternions*.

Scale

Objective: Change the objects size in on or more dimensions.



Objective: Change the objects size in on or more dimensions.

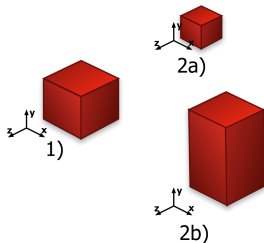


Given:

- Scale factors per dimension

$$s = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix}^T$$

Objective: Change the objects size in on or more dimensions.



Given:

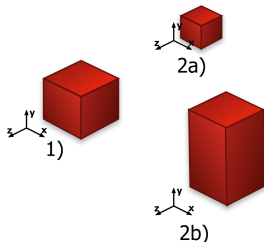
- Scale factors per dimension

$$s = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix}^T$$

- Point

$$\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix}^T$$

Objective: Change the objects size in on or more dimensions.



Given:

- Scale factors per dimension

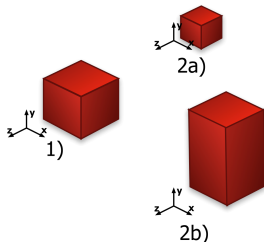
$$s = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix}^T$$

- Point

$$\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix}^T$$

$$p' = \begin{bmatrix} s_x p_x \\ s_y p_y \\ s_z p_z \\ 1 \end{bmatrix} = S(s_x, s_y, s_z) p = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Change the objects size in on or more dimensions.



Given:

- Scale factors per dimension

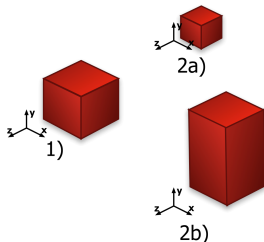
$$s = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix}^T$$

- Point

$$\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix}^T$$

$$p' = \begin{bmatrix} s_x p_x \\ s_y p_y \\ s_z p_z \\ 1 \end{bmatrix} = S(s_x, s_y, s_z) p = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Change the objects size in on or more dimensions.



Given:

- Scale factors per dimension

$$s = \begin{bmatrix} s_x & s_y & s_z \end{bmatrix}^T$$

- Point

$$\begin{bmatrix} p_x & p_y & p_z & 1 \end{bmatrix}^T$$

Affine Transform: Scale:

$$p' = \begin{bmatrix} s_x p_x \\ s_y p_y \\ s_z p_z \\ 1 \end{bmatrix} = S(s_x, s_y, s_z) p = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Properties of the Scale Matrix:

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Properties of the Scale Matrix:

- Effects both Models and Directions

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Properties of the Scale Matrix:

- ▶ Effects both Models and Directions
- ▶ Does not preserve shape²

Given:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

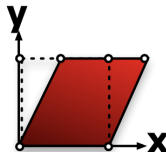
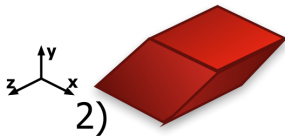
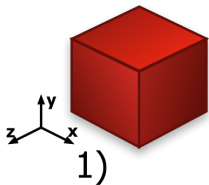
$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Properties of the Scale Matrix:

- ▶ Effects both Models and Directions
- ▶ Does not preserve shape²

²except if $s_x = s_y = s_z$

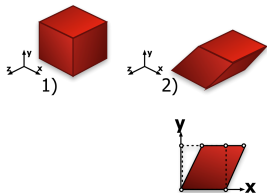
Objective: Shear an object along an axis.



Objective: Shear an object along an axis.

Required:

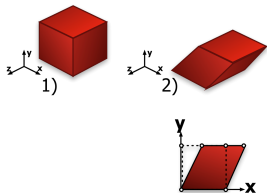
- Shear source $s \in x, y, z$



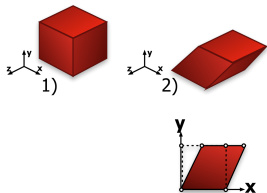
Objective: Shear an object along an axis.

Required:

- Shear source $s \in x, y, z$
- Shear target $t \in x, y, z$



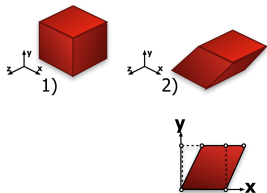
Objective: Shear an object along an axis.



Required:

- ▶ Shear source $s \in x, y, z$
- ▶ Shear target $t \in x, y, z$
- ▶ Shear proportionality constant σ_{st}

Objective: Shear an object along an axis.

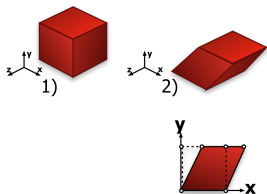


Required:

- ▶ Shear source $s \in x, y, z$
- ▶ Shear target $t \in x, y, z$
- ▶ Shear proportionality constant σ_{st}
- ▶ Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

Objective: Shear an object along an axis.

Required:

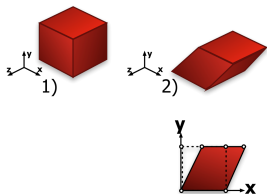


- ▶ Shear source $s \in x, y, z$
- ▶ Shear target $t \in x, y, z$
- ▶ Shear proportionality constant σ_{st}
- ▶ Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

For $s = x, t = y$, and hence σ_{xy}

Objective: Shear an object along an axis.

Required:



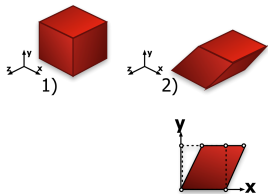
- Shear source $s \in x, y, z$
- Shear target $t \in x, y, z$
- Shear proportionality constant σ_{st}
- Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

For $s = x, t = y$, and hence σ_{xy}

$$p' = \begin{bmatrix} p_x + \sigma_{xy}p_y \\ p_y \\ p_z \\ 1 \end{bmatrix} = H_{xy}(\sigma_{xy})p = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Shear an object along an axis.

Required:



- ▶ Shear source $s \in x, y, z$
- ▶ Shear target $t \in x, y, z$
- ▶ Shear proportionality constant σ_{st}
- ▶ Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

For $s = x, t = y$, and hence σ_{xy}

$$p' = \begin{bmatrix} p_x + \sigma_{xy}p_y \\ p_y \\ p_z \\ 1 \end{bmatrix} = H_{xy}(\sigma_{xy})p = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Given:

$$H_{xy}(\sigma_{xy}) = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Given:

$$H_{xy}(\sigma_{xy}) = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$H_{xy}^{-1}(\sigma_{xy}) = H_{xy}(-\sigma_{xy})$$

Given:

$$H_{xy}(\sigma_{xy}) = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$\begin{aligned} H_{xy}^{-1}(\sigma_{xy}) &= H_{xy}(-\sigma_{xy}) \\ &= \begin{bmatrix} 1 & -\sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Properties of the Shear Matrix:

Given:

$$H_{xy}(\sigma_{xy}) = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$\begin{aligned} H_{xy}^{-1}(\sigma_{xy}) &= H_{xy}(-\sigma_{xy}) \\ &= \begin{bmatrix} 1 & -\sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Properties of the Shear Matrix:

- Effects both Models and Directions

Given:

$$H_{xy}(\sigma_{xy}) = \begin{bmatrix} 1 & \sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Follows:

$$\begin{aligned} H_{xy}^{-1}(\sigma_{xy}) &= H_{xy}(-\sigma_{xy}) \\ &= \begin{bmatrix} 1 & -\sigma_{xy} & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Properties of the Shear Matrix:

- Effects both Models and Directions
- Dose not preserves shape

- In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears
- ▶ Shears can be combined

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears
- ▶ Shears can be combined

Affine Transform: Shear:

$$H(\quad, \quad, \quad, \quad, \quad, \quad) = \begin{bmatrix} 1 & & & 0 \\ & 1 & & 0 \\ & & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears
- ▶ Shears can be combined

Affine Transform: Shear:

$$H(\sigma_{xy}, \sigma_{xz}, \quad, \quad, \quad, \quad) = \begin{bmatrix} 1 & \sigma_{xy} & \sigma_{xz} & 0 \\ & 1 & & 0 \\ & & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears
- ▶ Shears can be combined

Affine Transform: Shear:

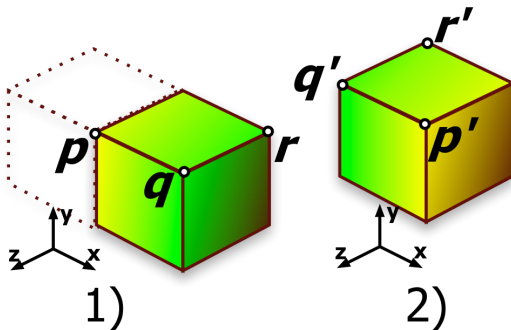
$$H(\sigma_{xy}, \sigma_{xz}, \sigma_{yx}, \sigma_{yz}, \quad , \quad) = \begin{bmatrix} 1 & \sigma_{xy} & \sigma_{xz} & 0 \\ \sigma_{yx} & 1 & \sigma_{yz} & 0 \\ & & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- ▶ In $\mathbb{R}^3$ there are 3 axis that can be combined with the other 2
- ▶ Therefore there are 6 source-target relations
- ▶ Results in 6 proportionality constants and 6 different shears
- ▶ Shears can be combined

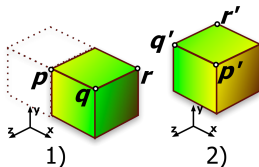
Affine Transform: Shear:

$$H(\sigma_{xy}, \sigma_{xz}, \sigma_{yx}, \sigma_{yz}, \sigma_{zx}, \sigma_{zy}) = \begin{bmatrix} 1 & \sigma_{xy} & \sigma_{xz} & 0 \\ \sigma_{yx} & 1 & \sigma_{yz} & 0 \\ \sigma_{zx} & \sigma_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Objective: Reflect an object at a plane, axis (line), or at a point.



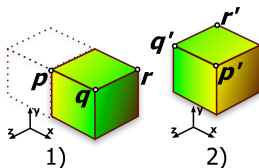
Objective: Reflect an object at a plane, axis (line), or at a point.



Required:

- Reflection relative to, here: yz -plane.

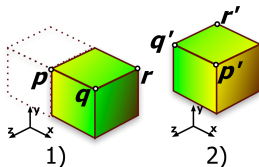
Objective: Reflect an object at a plane, axis (line), or at a point.



Required:

- ▶ Reflection relative to, here: yz -plane.
- ▶ Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

Objective: Reflect an object at a plane, axis (line), or at a point.

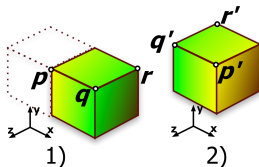


Required:

- Reflection relative to, here: yz -plane.
- Point $p = [p_x \ p_y \ p_z \ 1]^T$

$$p' = \begin{bmatrix} -p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} = F_{yz} p = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Reflect an object at a plane, axis (line), or at a point.

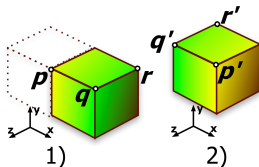


Required:

- Reflection relative to, here: yz -plane.
- Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

$$p' = \begin{bmatrix} -p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} = F_{yz} p = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Objective: Reflect an object at a plane, axis (line), or at a point.



Required:

- Reflection relative to, here: yz -plane.
- Point $p = [p_x \quad p_y \quad p_z \quad 1]^T$

Affine Transform: Reflection:

$$p' = \begin{bmatrix} -p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} = F_{yz} p = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

- Preserves the shape of an object

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions

Reflection Inverse

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale
- ▶ Is its own inverse

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale
- ▶ Is its own inverse

$$F_{yz}^{-1} =$$

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale
- ▶ Is its own inverse

$$F_{yz}^{-1} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} =$$

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale
- ▶ Is its own inverse

$$F_{yz}^{-1} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = F_{yz}^T =$$

- ▶ Preserves the shape of an object
- ▶ Effects models and Directions
- ▶ Is a negative scale
- ▶ Is its own inverse

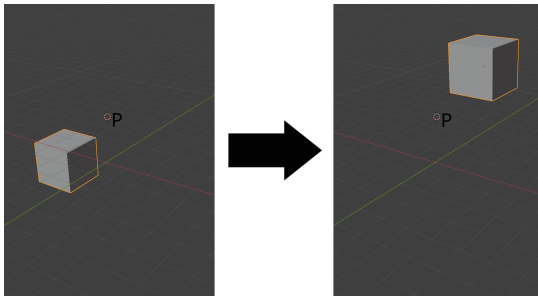
$$F_{yz}^{-1} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = F_{yz}^T = F_{yz}$$

Combining Transforms

Now we want to combine the transforms to mirror a shape around a point:

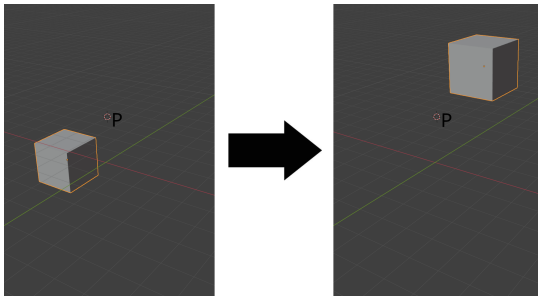
Combining Transforms

Now we want to combine the transforms to mirror a shape around a point:



Combining Transforms

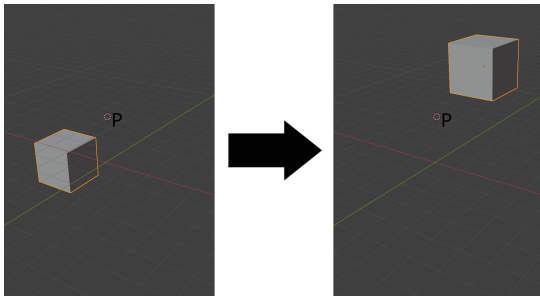
Now we want to combine the transforms to mirror a shape around a point:



$$a' = ? \cdot ? \cdot ? \cdot a$$

Combining Transforms

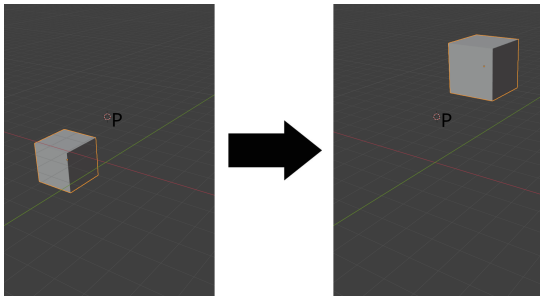
Now we want to combine the transforms to mirror a shape around a point:



$$a' = ? \cdot ? \cdot T(-p) \cdot a$$

Combining Transforms

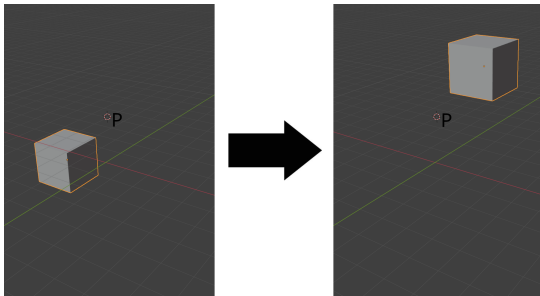
Now we want to combine the transforms to mirror a shape around a point:



$$a' = ? \cdot S(-1, -1, -1) \cdot T(-p) \cdot a$$

Combining Transforms

Now we want to combine the transforms to mirror a shape around a point:



$$a' = T(p) \cdot S(-1, -1, -1) \cdot T(-p) \cdot a$$

Preserves Shape and only effects points:

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis
- ▶ $E(p, y, r)$ - Rotate around all three axis

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis
- ▶ $E(p, y, r)$ - Rotate around all three axis
- ▶ F_{xy}, F_x, F_{xyz} - Mirror at a plane, axis, or at a point

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis
- ▶ $E(p, y, r)$ - Rotate around all three axis
- ▶ F_{xy}, F_x, F_{xyz} - Mirror at a plane, axis, or at a point

Changes Shape and effects points and directions:

Preserves Shape and only effects points:

- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis
- ▶ $E(p, y, r)$ - Rotate around all three axis
- ▶ F_{xy}, F_x, F_{xyz} - Mirror at a plane, axis, or at a point

Changes Shape and effects points and directions:

- ▶ $S(s_x, s_y, s_z)$ - Scale axis

Preserves Shape and only effects points:

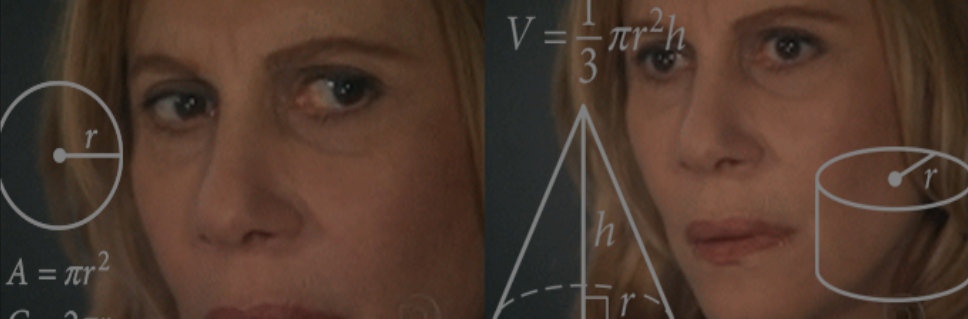
- ▶ $T(v_x, v_y, v_z)$ - Move point through space

Preserves Shape and effects points and directions:

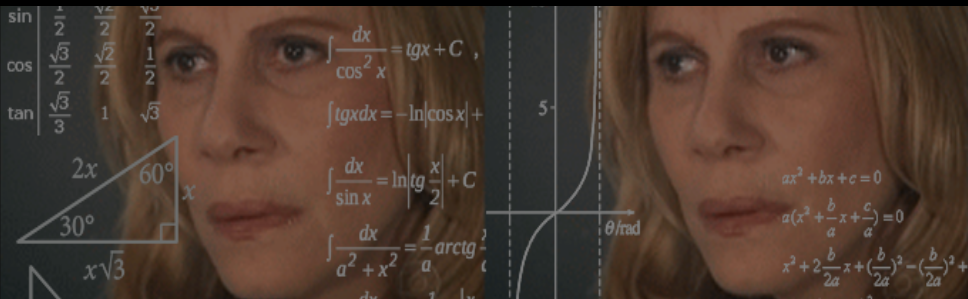
- ▶ $R_x(\delta), R_y(\delta), R_z(\delta)$ - Rotate around axis
- ▶ $E(p, y, r)$ - Rotate around all three axis
- ▶ F_{xy}, F_x, F_{xyz} - Mirror at a plane, axis, or at a point

Changes Shape and effects points and directions:

- ▶ $S(s_x, s_y, s_z)$ - Scale axis
- ▶ $H(\sigma_{xy}, \sigma_{xz}, \sigma_{yx}, \sigma_{yz}, \sigma_{zx}, \sigma_{zy})$ - Shear an axis about an other axis



Any Questions?

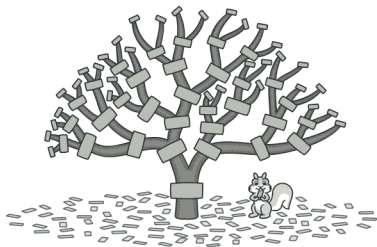


Composite

Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.

Composite

Composite is a structural design pattern that lets you compose objects into tree structures and then work with these structures as if they were individual objects.



World Space $\mathcal{W}$

Global reference system for all objects

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

Camera Space $\mathcal{C}$

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

Camera Space $\mathcal{C}$

- ▶ Local system of camera ($obj = cam$)

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

Camera Space $\mathcal{C}$

- ▶ Local system of camera ($obj = cam$)
- ▶ Eye at $\mathfrak{z}$, view dir is $-z$, up direction is y

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

Camera Space $\mathcal{C}$

- ▶ Local system of camera ($obj = cam$)
- ▶ Eye at $\mathfrak{z}$, view dir is $-z$, up direction is y
- ▶ Relation to $\mathcal{W}$ through **view transform** V

World Space $\mathcal{W}$

Global reference system for all objects

Object Space $\mathcal{O}_{obj}$

- ▶ Local system of each object obj
- ▶ Relation to $\mathcal{W}$ through **model transform** M

Camera Space $\mathcal{C}$

- ▶ Local system of camera ($obj = cam$)
- ▶ Eye at $\mathfrak{z}$, view dir is $-z$, up direction is y
- ▶ Relation to $\mathcal{W}$ through **view transform** V

Transforms change the reference coordinate system.

Idea: Hierarchical Scene Model

- Models usually are relative to their own local coordinate system

Idea: Hierarchical Scene Model

- ▶ Models usually are relative to their own local coordinate system
- ▶ Move them through space using a hierarchy of transforms.

Idea: Hierarchical Scene Model

- ▶ Models usually are relative to their own local coordinate system
- ▶ Move them through space using a hierarchy of transforms.
- ▶ Group some models together to reduce duplication

- ▶ Models usually are relative to their own local coordinates system
- ▶ Move them through space using a hierarchy of transforms.
- ▶ Group some models together to reduce duplication

⇒ **Hierarchical Scene Graph**

Idea: Hierarchical Scene Model

- ▶ Models usually are relative to their own local coordinates system
- ▶ Move them through space using a hierarchy of transforms.
- ▶ Group some models together to reduce duplication

⇒ **Hierarchical Scene Graph**

- ▶ Can contain additional data and behaviors

- ▶ Models usually are relative to their own local coordinates system
- ▶ Move them through space using a hierarchy of transforms.
- ▶ Group some models together to reduce duplication

⇒ **Hierarchical Scene Graph**

- ▶ Can contain additional data and behaviors
- ▶ Modeled as directed acyclic graph

Scene Graph: Node Types

We usually use two types of nodes:

Scene Graph: Node Types

We usually use two types of nodes:

- ▶ *Leaf Nodes*: contain renderable object

We usually use two types of nodes:

- ▶ *Leaf Nodes*: contain renderable object
- ▶ *Group Nodes*: have one ore more children:

We usually use two types of nodes:

- ▶ *Leaf Nodes*: contain renderable object
- ▶ *Group Nodes*: have one or more children:
 - Transformations

We usually use two types of nodes:

- ▶ *Leaf Nodes*: contain renderable object
- ▶ *Group Nodes*: have one ore more children:
 - Transformations
 - Materials

We usually use two types of nodes:

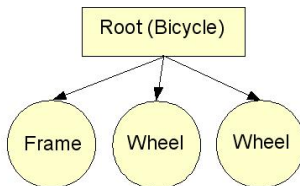
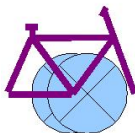
- ▶ *Leaf Nodes*: contain renderable object
- ▶ *Group Nodes*: have one or more children:
 - Transformations
 - Materials
 - Animation

We usually use two types of nodes:

- ▶ *Leaf Nodes*: contain renderable object
- ▶ *Group Nodes*: have one ore more children:
 - Transformations
 - Materials
 - Animation
 - Complex logic

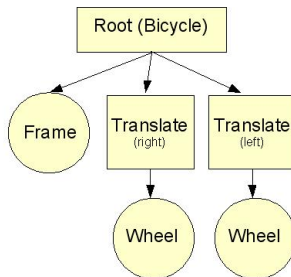
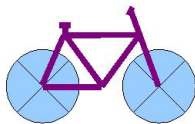
Scene Graph: Bicycle Example

Without a scene graph all our models are rendered at origin:



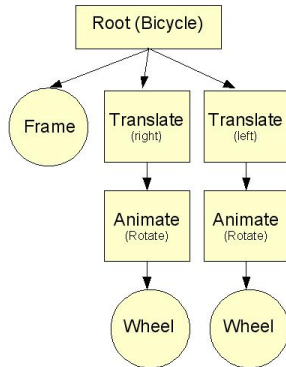
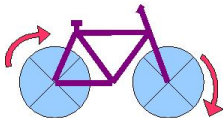
Scene Graph: Bicycle Example

With Translations the Bicycle comes into shape:



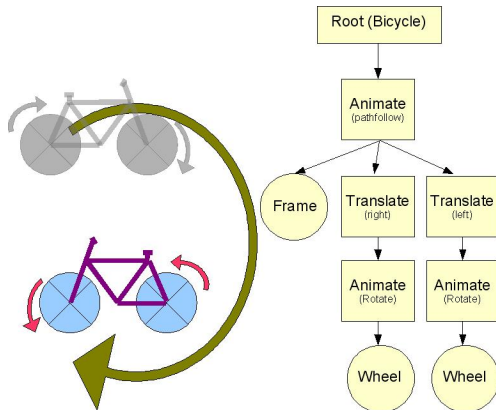
Scene Graph: Bicycle Example

Additional animation nodes (rotation over time) let the Bicycle come to life:



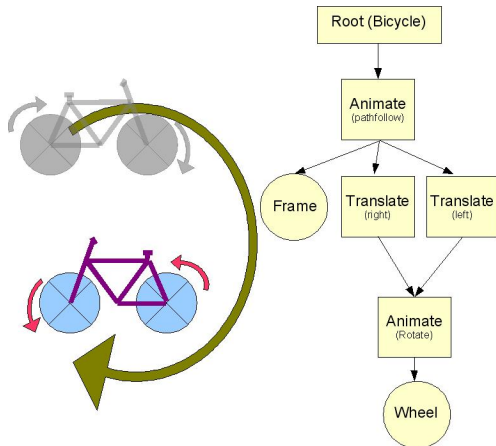
Scene Graph: Bicycle Example

Through a more complex animation node, the Bicycle follows a path:

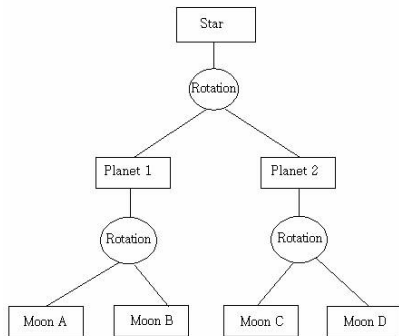


Scene Graph: Bicycle Example

To save memory we remove the duplication of the wheels:

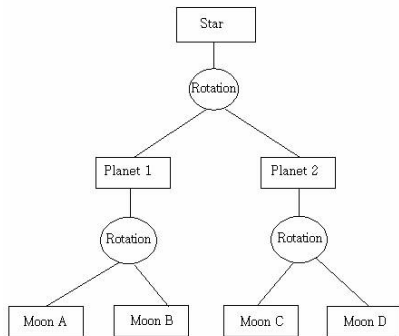


Scene Graph: Rendering Example



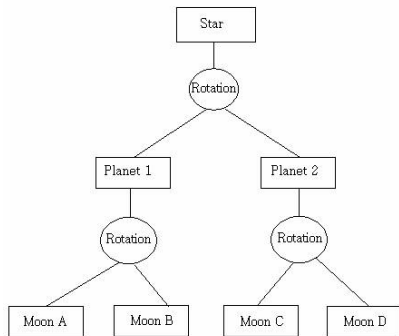
Scene Graph: Rendering Example

► Draw the star



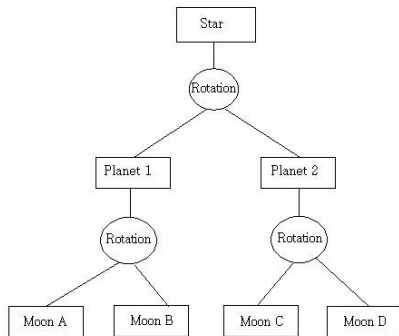
Scene Graph: Rendering Example

- Draw the star
- Save the current matrix



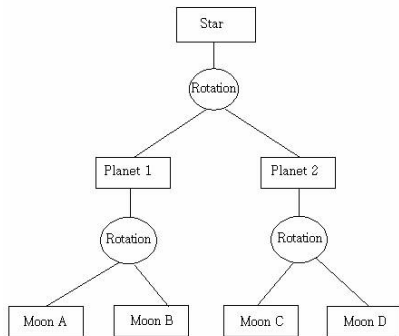
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation



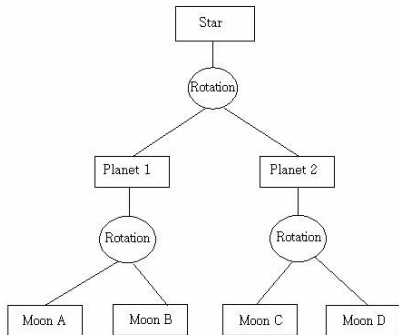
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One



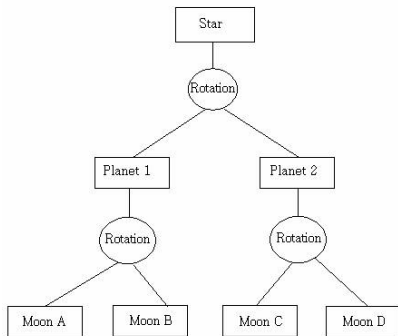
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix



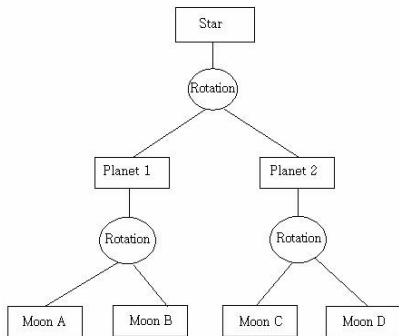
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation



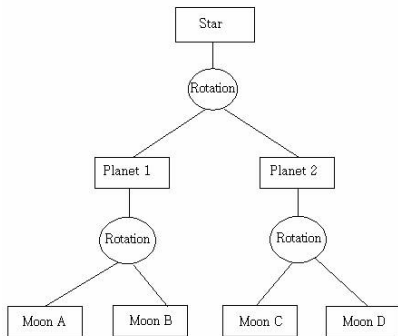
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A



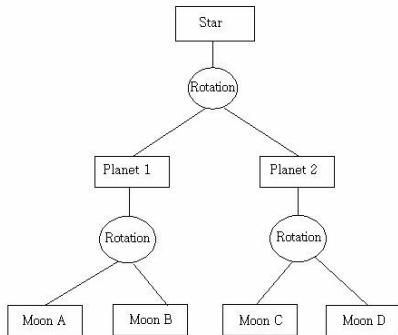
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B



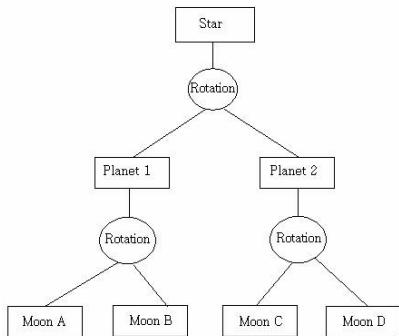
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved



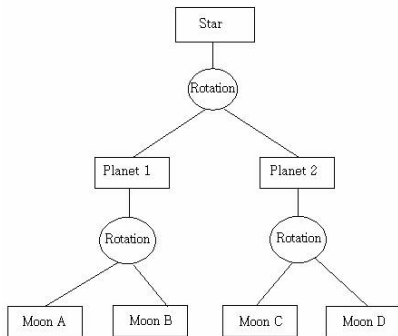
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One



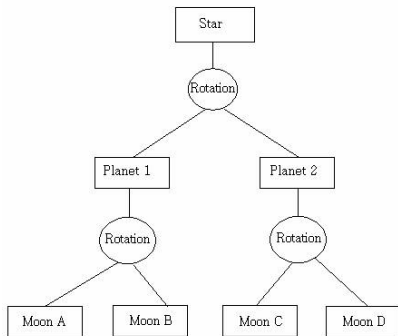
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix



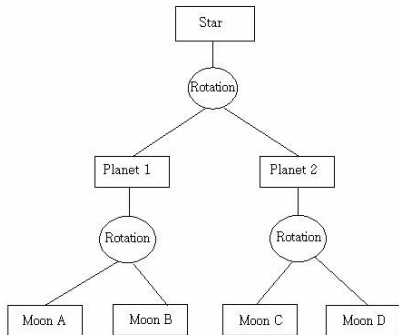
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix
 - Apply a rotation



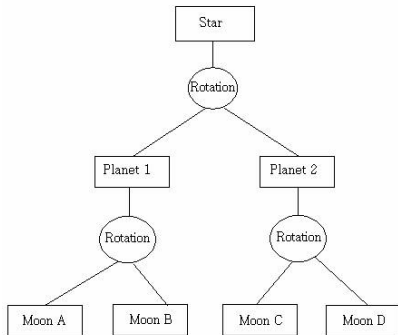
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix
 - Apply a rotation
 - Draw Moon C



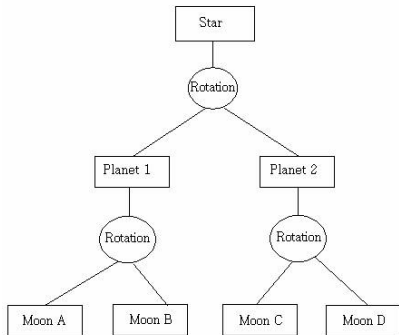
Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix
 - Apply a rotation
 - Draw Moon C
 - Draw Moon D

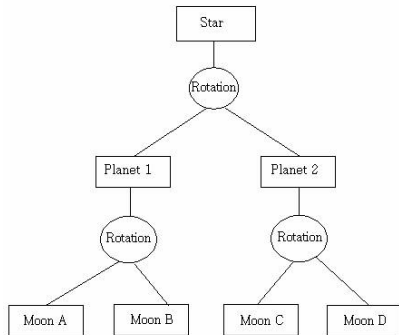


Scene Graph: Rendering Example

- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix
 - Apply a rotation
 - Draw Moon C
 - Draw Moon D
 - Reset the matrix we saved



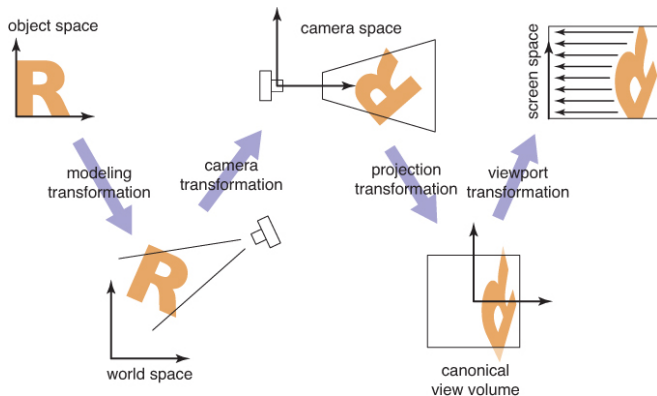
Scene Graph: Rendering Example



- ▶ Draw the star
- ▶ Save the current matrix
 - Apply a rotation
 - Draw Planet One
 - Save the current Matrix
 - Apply a second rotation
 - Draw Moon A
 - Draw Moon B
 - Reset the matrix we saved
 - Draw Planet One
 - Save the current Matrix
 - Apply a rotation
 - Draw Moon C
 - Draw Moon D
 - Reset the matrix we saved
- ▶ Reset the matrix we saved

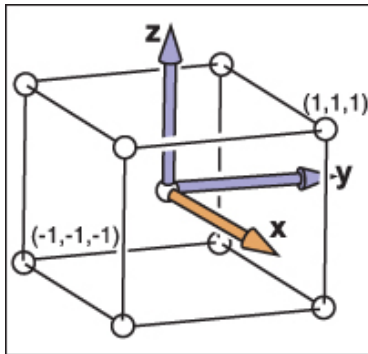
Projection Matrix

To get to the final transform for rendering we still need to account for projection:



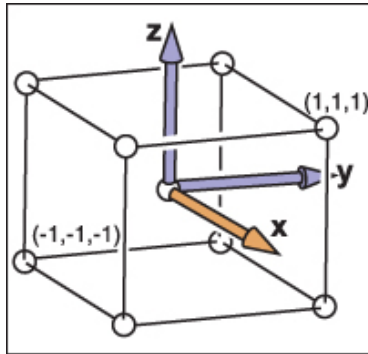
Canonical View Volume

The canonical view volume is a cube with side of length two centered at the origin:



Canonical View Volume

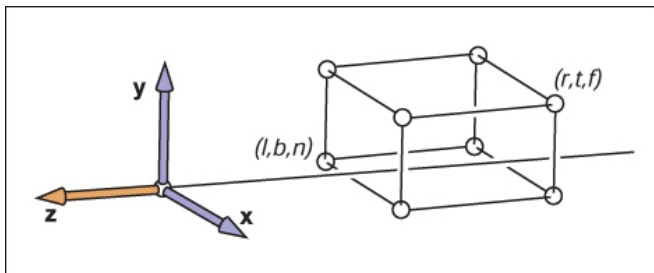
The canonical view volume is a cube with side of length two centered at the origin:



We need to transform our coordinate space in a way, such that everything we want to render is inside of this cube.

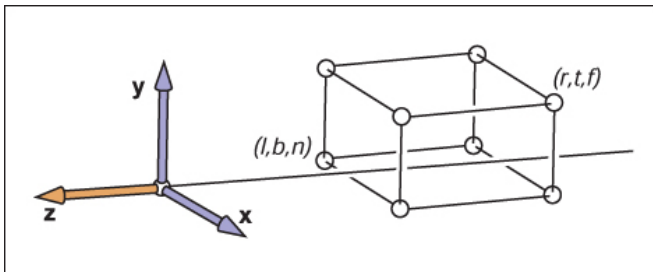
Orthographic View

The ortho-graphic view volume:



Orthographic View

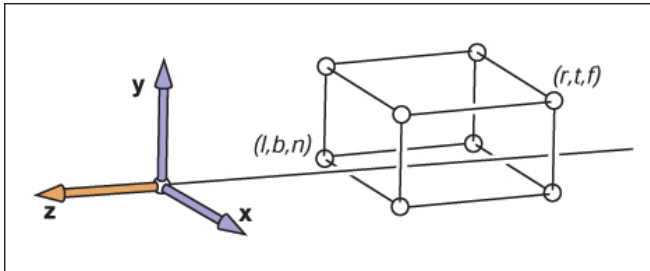
The ortho-graphic view volume:



From the different planes we can construct a view matrix:

Orthographic View

The ortho-graphic view volume:



From the different planes we can construct a view matrix:

$x = l \equiv$ left plane,

$x = r \equiv$ right plane,

$y = b \equiv$ bottom plane,

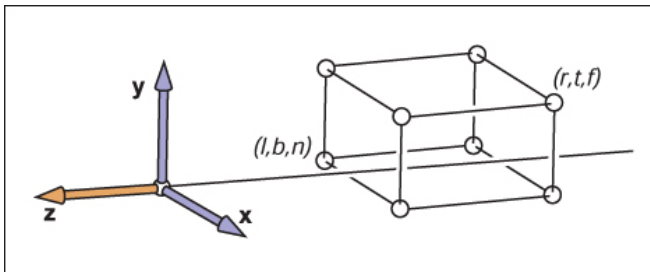
$y = t \equiv$ top plane,

$z = n \equiv$ near plane

$z = f \equiv$ far plane.

Orthographic View

The ortho-graphic view volume:



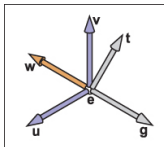
From the different planes we can construct a view matrix:

$x = l \equiv$ left plane,
 $x = r \equiv$ right plane,
 $y = b \equiv$ bottom plane,
 $y = t \equiv$ top plane,
 $z = n \equiv$ near plane
 $z = f \equiv$ far plane.

$$M_{\text{orth}} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The Camera Transformation

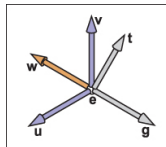
We construct the camera transform from:



The Camera Transformation

We construct the camera transform from:

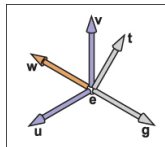
- the eye position **e**



The Camera Transformation

We construct the camera transform from:

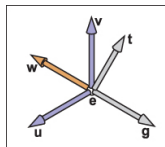
- ▶ the eye position **e**
- ▶ the gaze direction **g**



The Camera Transformation

We construct the camera transform from:

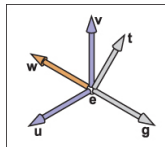
- ▶ the eye position **e**
- ▶ the gaze direction **g**
- ▶ the view-up vector **t**



The Camera Transformation

We construct the camera transform from:

- ▶ the eye position **e**
- ▶ the gaze direction **g**
- ▶ the view-up vector **t**

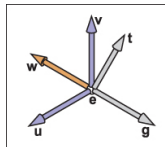


▶ $\mathbf{w} = -\mathbf{g}$

The Camera Transformation

We construct the camera transform from:

- ▶ the eye position **e**
- ▶ the gaze direction **g**
- ▶ the view-up vector **t**

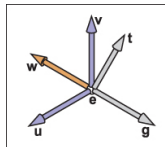


- ▶ **w = -g**
- ▶ **v** = same plane as **g** and **t**

The Camera Transformation

We construct the camera transform from:

- ▶ the eye position **e**
- ▶ the gaze direction **g**
- ▶ the view-up vector **t**

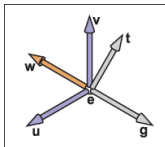


- ▶ **w = -g**
- ▶ **v = same plane as g and t**
- ▶ **u = w × v**

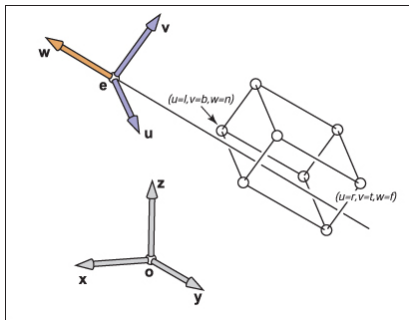
The Camera Transformation

We construct the camera transform from:

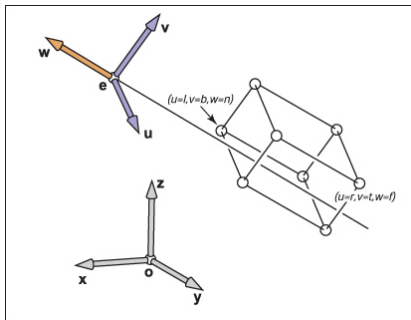
- ▶ the eye position $\mathbf{e}$
- ▶ the gaze direction $\mathbf{g}$
- ▶ the view-up vector $\mathbf{t}$



- ▶ $\mathbf{w} = -\mathbf{g}$
- ▶ $\mathbf{v}$ = same plane as $\mathbf{g}$ and $\mathbf{t}$
- ▶ $\mathbf{u} = \mathbf{w} \times \mathbf{v}$

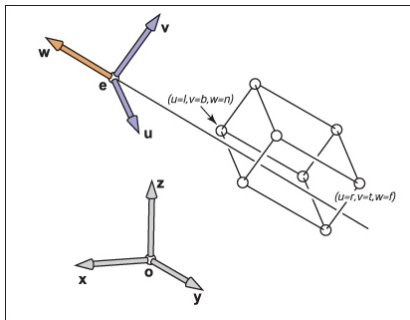


The Camera Transformation Matrix



Exact definition for **w,v,u**:

The Camera Transformation Matrix



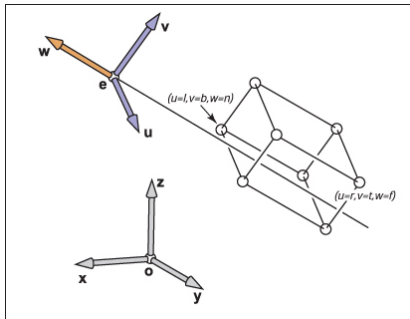
Exact definition for **w,v,u**:

$$\mathbf{w} = -\frac{\mathbf{g}}{\|\mathbf{g}\|},$$

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|},$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}.$$

The Camera Transformation Matrix



Exact definition for **w,v,u**:

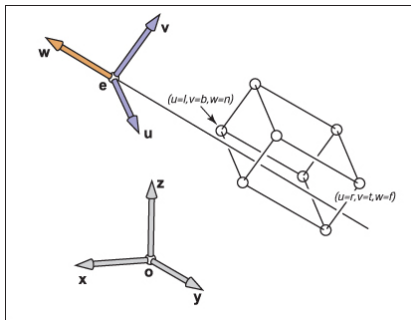
$$\mathbf{w} = -\frac{\mathbf{g}}{\|\mathbf{g}\|},$$

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|},$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}.$$

We move the origin to **e**

The Camera Transformation Matrix



Exact definition for **w,v,u**:

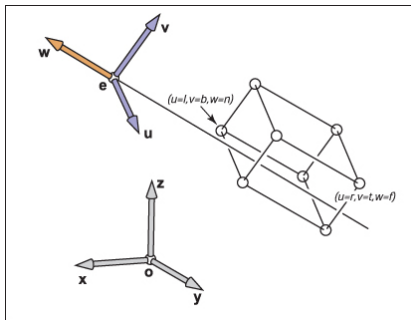
$$\mathbf{w} = -\frac{\mathbf{g}}{\|\mathbf{g}\|},$$

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|},$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}.$$

We move the origin to **e** and align the axis **x,y,z** with **w,u,v**:

The Camera Transformation Matrix



Exact definition for **w,v,u**:

$$\mathbf{w} = -\frac{\mathbf{g}}{\|\mathbf{g}\|},$$

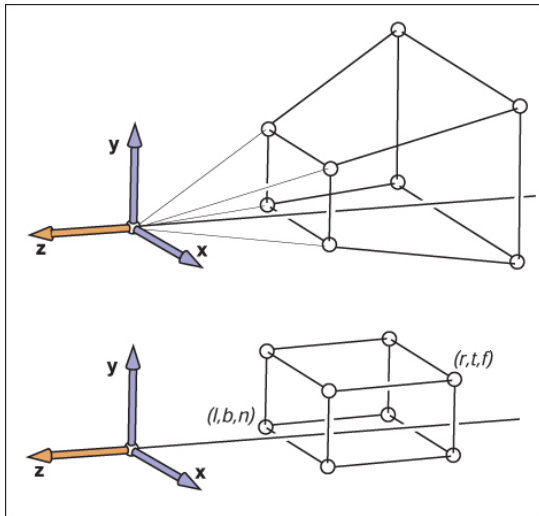
$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|},$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}.$$

We move the origin to **e** and align the axis **x,y,z** with **w,u,v**:

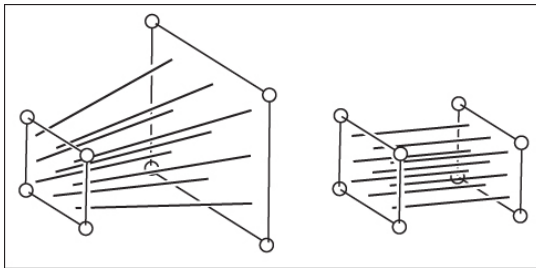
$$M_{\text{cam}} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{w} & \mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Perspective Projection



Perspective Projection

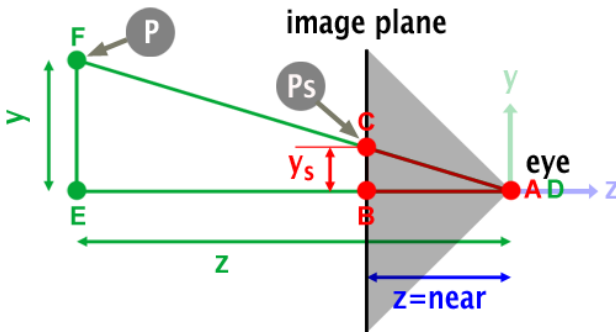
In orthogonal projections the lines between near and far plane are parallel:



In perspective projection the lines between near and far plane diverge.

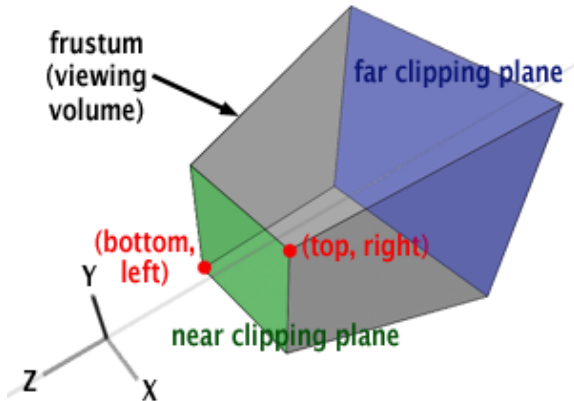
Perspective Projection

Lets have a look a the projection pyramid from the side profile:



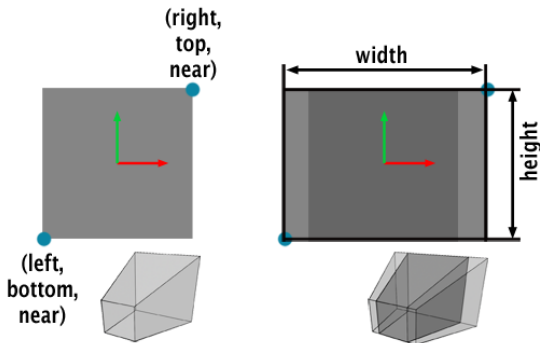
Perspective Projection

The pyramid is clipped at the front by the near plane and clipped at the end by the far plane:



Perspective Projection

We also need to adjust the frustum to account for aspect ratio:



Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

- ▶ z_{Near} = distance from the eye **e** and the image plane

Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

- ▶ z_{Near} = distance from the eye **e** and the image plane
- ▶ z_{Far} = distance from the eye **e** and far clipping plane

Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)

Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)
- ▶ $aspect = \frac{viewWidth}{viewHeight}$

Perspective Projection Matrix

The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)
- ▶ $aspect = \frac{viewWidth}{viewHeight}$
- ▶ $f = \cotangent(\frac{fovy}{2})$

The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)
- ▶ $aspect = \frac{viewWidth}{viewHeight}$
- ▶ $f = \cotangent(\frac{fovy}{2})$

Therefore³ the projection matrix is defined as:

The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)
- ▶ $aspect = \frac{viewWidth}{viewHeight}$
- ▶ $f = \cotangent(\frac{fovy}{2})$

Therefore³ the projection matrix is defined as:

$$P = \begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{zFar+zNear}{zNear-zFar} & \frac{2 \times zFar \times zNear}{zNear-zFar} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

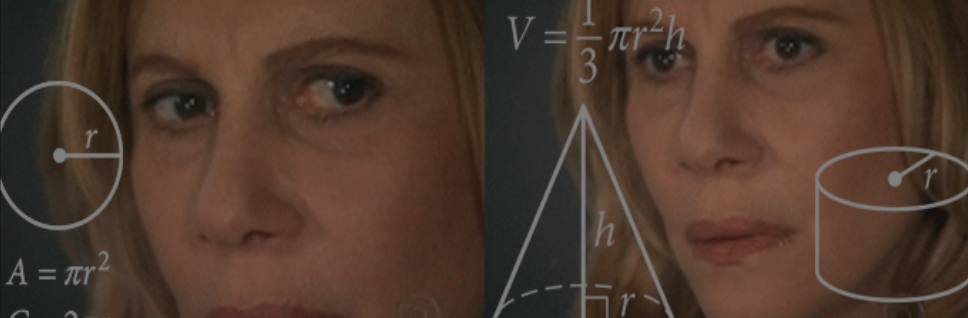
The Perspective Projection Matrix can be calculated from:

- ▶ $zNear$ = distance from the eye **e** and the image plane
- ▶ $zFar$ = distance from the eye **e** and far clipping plane
- ▶ $fovy$ = Field of view angle (y)
- ▶ $aspect = \frac{viewWidth}{viewHeight}$
- ▶ $f = \cotangent(\frac{fovy}{2})$

Therefore³ the projection matrix is defined as:

$$P = \begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{zFar+zNear}{zNear-zFar} & \frac{2 \times zFar \times zNear}{zNear-zFar} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

³The proof and derivation of the projection matrix can be read in the relevant Literature.



Any Questions?

